DECLARATION

I **Aryan Bhargav** and **Kislay Dubey** student of B.Tech (5CSE1-X) hereby declare that the project titled **"Malware Detection in Computer"** which is submitted by us to **Mr. Sanjeev Thakur** (Head Department of Computer Science and Engineering), Amity School of Engineering Technology, Amity University Uttar Pradesh, Noida, in partial fulfilment of requirement for the award of the degree of Bachelor of Technology in Computer Science and Engineering, has not been previously formed the basis for the award of any degree, diploma or other similar title or recognition and faithful record of work carried out by him under my guidance and supervision. To the best of my knowledge, this work has not been submitted in part or in full for any degree or diploma at this university or elsewhere.

(Signature of the Guide)

Date: 02/05/2025

Mr. Sanjeev Thakur (HOD)

(Faculty Guide) ASET,

Noida.

CERTIFICATE

This is to certify that **Mr. Kislay Dubey** and **Mr. Aryan Bhargav** student of B. Tech in Computer Science and Engineering has carried out work presented in the project of the Term paper entitle "Malware Detection in Computers" as a part of Second year program of Bachelor of Technology in Computer Science and Engineering from Amity University, Uttar Pradesh, Noida under my supervision.

_____

Mr. Sanjeev Thakur (HOD)

Head Of Department of Computer Science and Engineering

ASET, Noida

ACKNOWLEDGEMENT

The completion of this task would be incomplete without my teammate, whose ceaseless cooperation made it possible. I want to thank **Mr. Sanjeev Thakur, Head of the Department of CSE**, and Amity University for providing me with the opportunity to undertake this project. I would like to thank my faculty guide, **Mr. Sanjeev Thakur**, who has always been there to solve queries and guide our team in the right direction regarding the project. Without her help and inspiration, I would not have been able to complete the project.

(Signature of the Student)

Date: 02/05/2025

Kislay Dubey

(A2305223028)

ABSTRACT

Malware Detection is a critical area in the field of cybersecurity, dedicated to identifying, analysing malicious software designed to disrupt, damage, or gain unauthorized access to computer systems. Malware Detection is a modern engineering science focused on identifying and preventing malicious software that can harm computer systems. Over the years, companies such as Google, Microsoft, Cisco, etc. have invested in the improvement of cybersecurity solutions. Detecting malicious content in software remains complex, and its scope is vast. This field brings knowledge and methods from Computer Science, Machine Learning, Data Science, Network Security, etc. Traditional malware detection is based on signature-based techniques, where patterns of malicious code are used to identify threats. These methods struggle against known malware and are unable to detect new malware.

The use of Machine Learning (ML) in malware detection has increased the rate of detecting malware inside the computer. With the help of the datasets, graphs, API calls, file entropy, access patterns, and network activity, malware can be detected inside the computer. Dynamic analysis observes the program behaviour in a controlled environment such as a sandbox, whereas static analysis examines code without execution. Both methods help extract features for machine learning models.

In conclusion, malware detection is a rapidly evolving field essential for protecting digital systems. With the rise of sophisticated cyber threats, machine learning has become a critical tool for identifying and preventing attacks. Ongoing research and collaboration are key to staying ahead in the fight against malware.

INDEX

1. Introduction

1.1 Purpose/Motivation
1.2 Objective

2. Literature Review

3. Methodology

4. Implementation and Analysis
4.1 Data Visualization and Insights

INTRODUCTION

In today's world, every system is at risk of being attacked by malicious software. Cyber criminals now have access to automated technologies that are more powerful, and new dangers surface every second. Because of this, maintaining adequate cybersecurity might be difficult.

In today's digital world, one of the most important challenges is to detect malicious software, and unfortunately, the situation is only becoming worse.

Malware or malicious software is a software that is programmed to cause damage to a user's computer without the user's permission. Hackers deploy this malware in the form of emails, fake websites, or files to attract the user's attention to open or download it. Malware can come in different forms, such as viruses, worms, trojans, ransomware, spyware, adware, and rootkits, which can corrupt and steal sensitive information of the user. Its primary targets are computational platforms, IoT devices, medical equipment, and E&I control systems. These platforms include embedded computing devices. Malware is characterized by its complexity and its ability to change not only the code but also its behaviour to prevent detection. Because there is such a vast range of malicious software in circulation, it is not sufficient to rely just on protections that are based on signatures; rather, a more complete toolbox is necessary.

This NTCC project aims to explore malware detection mechanisms with a focus on machine learning models like Random Forest classifiers, their feature extraction methods, and how they distinguish between malicious and benign software.

TYPES OF MALWARE

1. Viruses:

- Characteristics: Viruses are self-replicating code that attaches itself to legitimate programs or files. It requires user interaction to spread.
- Impact: It can modify, corrupt, or delete files, disrupt system functionality, and spread to other devices.

2. Worms:

- Characteristics: Worms are self-replicating standalone programs that spread across networks without user interaction.
- Impact: Worms can consume network resources, slow down systems, and help in the distribution of other malware.

3. Trojan Horses:

- Characteristics: Trojan is a type of malicious software that disguises itself as a legitimate or harmless program to enter a user's computer. It is dependent on social engineering to trick users into downloading and executing them.
- Impact: It can steal data (including financial information and credentials), gain unauthorized access, and delete or modify files.

4. Ransomware:

- Characteristics: It can encrypt a victim's files and lock them out of their device, then demand a ransom payment in digital currency in exchange for decryption or accessing the data.
- Impact: Data loss, significant financial costs, and business disruption.

5. Spyware:

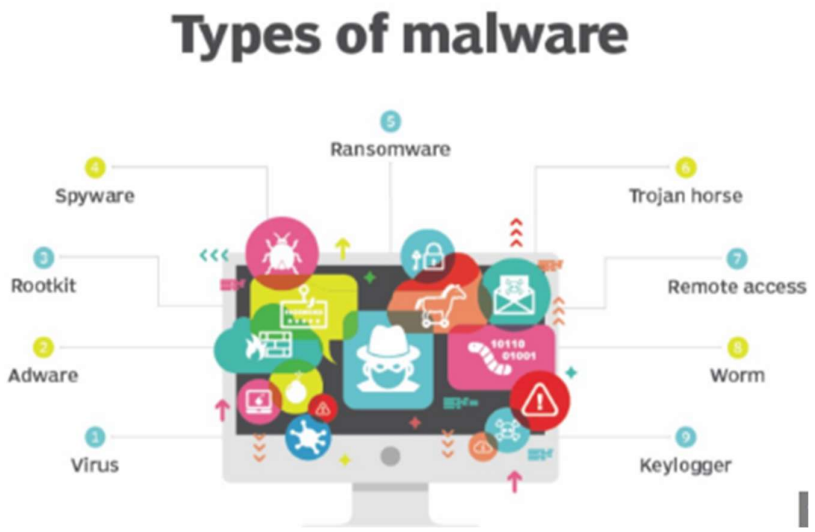
- Characteristics: It secretly monitors user activities and collects sensitive information without the user's knowledge or consent.
- Impact: Privacy invasion, identity theft, and potential financial fraud.

6. Adware:

- Characteristics: The purpose of adware is to display unwanted ads, often in the form of pop-ups, banners, or redirects to advertising websites.
- Impact: Slows the system performance and the potential collection of user data for targeted advertising.

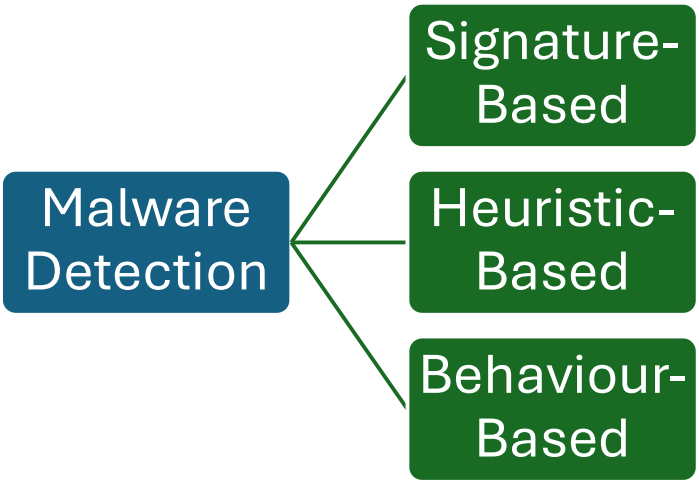
7. Rootkits:

- Characteristics: A stealthy type of malware that deeply embeds itself within a computer's operating system, providing root-level access to attackers while hiding its presence from conventional security scans.
- Impact: Long-term, hidden access for attackers, making detection and removal extremely difficult. Can disable antivirus software.



TYPES OF MALWARE DETECTION

Malware detection is an important part of cybersecurity. It aims to identify and prevent the spread of malware before it harms the computer. There are various techniques for detecting malware, which are broadly categorized into three main types: signature-based detection, heuristic-based detection, and behaviour-based detection.



Signature-Based Detection

It is one of the earliest and most widely used techniques to detect malware in a computer. This method uses a unique characteristic or "signatures" to identify the patterns of malware in the computer.

Malware Signature: It is a unique digital fingerprint or pattern associated with a specific piece of malware. It includes:

- **Hash values:** Hash values are a unique, fixed-size string of characters generated from a file's content; if a single bit in the file changes, the hash value will change. **Example:** A text file having **"Hello World"** will have a hash value of **SHA-256.**

- **Specific code patterns/byte sequences:** These unique sequences of bytes or instructions are found within the malware executable code.

Example: E8 00 00 00 00 5B 81 EB 78 56 34 12, in this sequence, the
pattern 81 **EB 78 56 34 12 – SUB EBX, 0x12345678,** a common trick in malware to adjust pointers or change the memory addresses.

- **File names and locations:** Certain file names or install themselves in specific directories by changing the alphabet in the file name with a number through typo-squatting. For example, **File Name:** svch0st.exe, **Location:** C:\Windows\System32, malware uses these types of file names to enter the user's computer without getting noticed.

WORKING

Malware Identification by Researchers

When a new malware is discovered via a user report, a honeypot trap, or an automated system. Cybersecurity analysts or automated tools analyze the malicious file. They investigate:

- Binary content (machine code).
- File structure.
- Strings and embedded commands.
- API calls and instructions.
- Code behaviour.

Creating a Signature

After analysing the malware, a signature is generated, which is a unique identifier of the malware. It can be:

- Static Byte Sequence: A fixed pattern in the binary code (e.g., E8 3A 00 FF).
- Hash Value: A cryptographic hash of the file, like MD5, SHA-1, or SHA-256.
- String Signatures: Unique text strings embedded in malware like URLs, commands, registry paths, etc.

Scanning System Files

The antivirus scans the entire system, and its functions are:

- It **does not execute** the file.
- It reads the file content and structure directly.
- It **compares** each file against the known malware signatures from the database.

Signature Match = Malware Detected

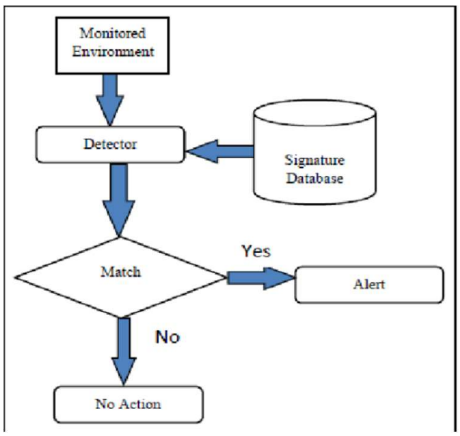
If a file matches a signature in the database, it is termed as **malicious**. The antivirus will then **take action.** If **no match** is found, the file is considered safe.

Advantages

1. Quick scan of files without executing them.
2. Very accurate for known threats.
3. Simple comparison algorithm.

Limitations

1. No protection against new malware.
2. Malware generators can use **polymorphism** to avoid signature matching.
3. Requires **frequent updates** to stay effective.

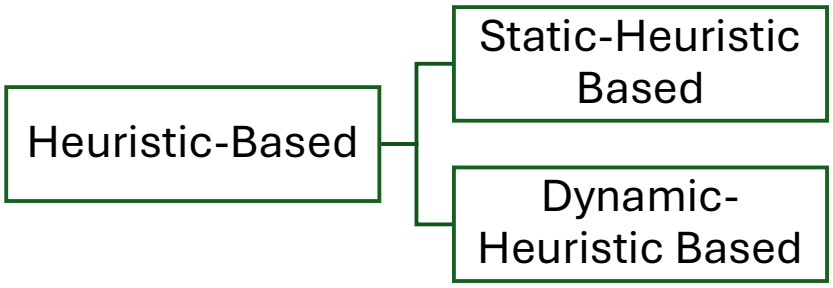


Heuristic-Based Detection

It is a malware detection technique that identifies **unknown or new types of malware** based on **suspicious behaviour, structure, and properties**. It does not rely on exact signatures used in **signature-based detection**. It's useful in catching new malware that hasn't been identified yet.

WORKING

Heuristic-Based Detection works in two modes:



Static Heuristic Analysis: This type of analysis scans the code for rules or patterns based on how malware typically behaves. It identifies any anomalies in the code that resemble known threats, as well as any malware that has never been seen before.

WORKING

1. **Examination of the code:** The suspicious program or file is first analysed to extract its code. This might involve decompiling the program back to its source code.

2. **Feature Extraction and Analysis:** The static heuristic engine then analyzes various features and characteristics of this extracted code.to find the suspicious properties or patterns that are commonly found in malware, like:
    - Obfuscated/encrypted strings (**E8 00 00 00 00 5B 81 EB 78 56 3412)**
    - Calls to dangerous Windows APIs (**File Name:** svch0st.exe, **Location:** C:\Windows\System32)
3. **Pattern Matching**: These patterns are compared against a set of predefined databases containing "malicious patterns." These rules are often based on observations of how this malware behaves.

4. **Thresholding and Flagging:** If the heuristic score exceeds a predefined threshold, the file is termed as malicious.

Dynamic Heuristic Analysis: This analysis is a powerful technique used to detect malicious behaviour during the actual execution of a program, unlike static heuristic analysis, which examines code without running it, detecting unknown or evolving threats by analyzing their behaviour rather than relying on predefined signatures.

The key features of dynamic heuristic analysis are:

- **Behavioural Observation:** It monitors a program's actions, such as file modifications, network connections, memory usage, and API calls.

- **Controlled Environment:** This is usually done in a virtual machine, an isolated environment to prevent any potential malicious activity.

- **Detection of Unknown Threats:** Its primary strength is in identifying "zero-day" threats (brand new, unseen malware) and polymorphic malware (which constantly changes its code to prevent signature-based detection).

WORKING

Let's take an example of a file named **"freemoney.exe".**

1. Isolated Room (Sandbox): Running the suspicious file on an actual computer, it is executed in a fake computer lab**.**

- **Example:** User gets an email of a file named **"freemoney.exe".** The security software will not open it on a real computer. Instead, this file is sent to a special computer ("sandbox") that's completely cut off from your real system.

2. Execution: The suspicious file to run inside the fake computer.

- **Example:** In the sandbox, **"freemoney.exe"** starts up, just like it would on a normal computer.

3. Behavioural monitoring: Sandbox monitors the suspected file in ways like:

- **Messing with the files:** Like encrypting them or deleting them.
- **Changing the computer's rules:** So, it can start up on its own.
- **Talking to bad websites:** Trying to send out your info or get more instructions.
- **Using hidden computer tricks:** Performing actions in a way only malware would.

4. Evaluation: Comparing what the file did to a list of known "bad behaviours" or suspicious patterns.

- If a program encrypts a bunch of files and deletes the originals, that's ransomware.
- If a program tries to turn off your antivirus, it's suspicious.

This is how Dynamic Heuristic Analysis works in detecting malicious files in a computer.

Behaviour-Based Detection.

It is a dynamic technique used in cybersecurity to detect malware by what it *does*. This technique focuses on monitoring the behaviour of a program during execution, rather than analysing it in a real-time or sandbox environment. It is especially powerful for detecting new, unknown, or polymorphic malware that can evade signature-based and heuristic methods.

HOW TO EFFECTIVELY IMPLEMENT IT

1) **Continuous Threat Intelligence Monitoring:** Monitor threat intelligence feeds to discover new attack techniques and threat activities.

2) **Detailed Requirement Gathering:** Gather intelligence, which deeply analyzes the attack's methodology to understand the core behaviour that needs to be detected.

3) **Rigorous Development and Testing:** Write detection code for identified behaviours and thoroughly test it in a controlled environment to ensure accuracy and minimize false positives before deployment.

4) **Adapt to Organizational Risk Tolerance:** Adjust the stringency (wider or narrower net) of detections based on your organization's specific risk appetite and resource availability.

5) **Continuous Re-evaluation:** Regularly assess and adapt your behaviour-based detection strategy to the ever-evolving threat landscape and changing organizational priorities.

WORKING

1. Program Execution: A file or application is run on the system. Instead of just scanning its code, the system starts monitoring the behaviour of the file in real time.

2.Monitoring the file: The behaviour-based system watches what the program performs:

- **System calls: Example: CreateFile with suspicious attributes:** If a process is observed repeatedly trying to create files with unusual extensions, this might signal ransomware activity.

- **Network Connections:** If MS Word suddenly tries to establish an SMB (Server Message Block) connection to another internal host, or an HTTP/HTTPS connection to a newly registered domain, it's highly anomalous behaviour.

How a behavior-based system watches it: The system intercepts or hooks these calls as they happen, recording details like the process making the call, the type of call, and its arguments.

3. Change in the file system: If a process rapidly renames numerous user files (e.g., .docx to .docx, encrypt or. ransoms) and writes new, encrypted versions, this is a hallmark of ransomware.

Changes to files like lsass.exe, winlogon.exe, or DLLs in System32 by non-system processes can indicate patching, hooking, or replacement by malware.

How a behavior-based system watches it: Monitors file access events, recording the process involved, the file path, the type of operation (read, write, delete), and the user context.

4. Editing the registry: Changes made to the Windows Registry, a hierarchical database that stores low-level settings for the operating system. It is a common target for malware persistence and configuration.

How a behaviour-based system watches it: Monitors reads, writes, and deletions of registry keys and values, along with the process and user acting.

Examples of suspicious registry edits monitored:

- **Adding entries to Run/RunOnce keys:** A program modifying **HKCU\Software\Microsoft\Windows\CurrentVersion\Run** or **HKLM\Software\Microsoft\Windows\CurrentVersion\Run** to ensure it starts automatically with Windows, if the entry points to an unusual location executable.
- **Disabling security features:** Modifications to keys related to Windows Defender **(HKEY_LOCAL_MACHINE\SOFTWARE\Policies\Microsoft\ Windows Defender), firewall rules, or** User Account Control (UAC) settings to weaken system defences.

5. Process Creation: The act of a malicious program launching another. malicious activity on the computer.

How a behaviour-based system watches it: Monitors parent-child process relationships, command-line arguments, process hashes, and user context.

Examples of suspicious process being monitored:

- **Unusual parent-child relationships:**

  Word.exe spawning **cmd.exe, powershell.exe, or certutil.exe.** (A document automatically opening a command prompt is highly suspicious.)

  Execution from suspicious locations: A process launching an executable from a temporary folder **(C:\Windows\Temp\, C:\Users\<user>\AppData\Local\Temp\)**.

1.1 PURPOSE

This study is motivated by the rise of complex cyber threats that traditional malware detection methods struggle to handle. To address this, it explores the use of Machine Learning, which can learn from data, adapt to new threats, and enhance real-time malware detection. The research also contributes to cybersecurity by demonstrating how data-driven techniques can strengthen digital protection.

1.2 OBJECTIVE

The objective of this project is to develop a Machine Learning (ML) based system that detects malware inside the computer. This involves a labelled dataset that contains both static and dynamic features, such as file size, entropy, number of imports, API calls, and network activity, to train and evaluate machine learning models. Supervised machine learning algorithms, including Random Forest and Support Vector Machine (SVM), are applied to build classification models that distinguish between benign and malicious files. Model performance is assessed using key evaluation metrics such as accuracy, precision, recall, F1-score, and ROC-AUC.

LITERATURE REVIEW

List of research done by various professors on malware:

[1] Arkajit Datta (VIT) and Kakelli Anil Kumar (VIT) made a paper on **"An Emerging Malware Analysis Techniques and Tools"**. The paper presents a comparative study of malware analysis techniques—**static**, **dynamic**, and **machine learning-based**—along with commonly used **cloud-based tools** like Virus Total, Valkyrie, and Cognitive Sandboxing. It finds that **dynamic analysis** is more reliable than static methods, especially against obfuscated malware. They proposed a **three-phase framework**: start with static analysis, use dynamic tools if needed, and apply **ML techniques** for high accuracy and adaptability.

[2] Rami Sihwail, Khairuddin Omar made a report "**A Survey on Malware Analysis Techniques: Static, Dynamic, Hybrid and Memory Analysis".** This report states that there are four types of analysis: static, dynamic, hybrid, and memory analysis. Static analysis inspects malware code without execution, is fast, but weak against obfuscation. Dynamic analysis runs malware in a sandbox to observe real behaviour. I is effective but resource-heavy. Hybrid analysis combines both for better accuracy. Memory analysis focuses on RAM to detect stealthy or fileless malware using tools like Volatility.

[3] Mohammed Mansoor Ali, Mohd Sahil Ali, Saad Mohammed Mohi Uddin, and Dr. Md Zainlabuddi from ISL Engineering College (HYD) researched **"Optimized Multi-Model Detection of Android Malware"**. The paper introduces **AAMD-OELAC**, which is an optimized ensemble learning framework designed for Android malware detection. This approach combines multiple machine learning models, selectively chosen through an evolutionary chi-square feature selection algorithm, to identify the most

discriminatory characteristics across Android apps—such as API calls, permissions, and code patterns.

[4] Ioannis Dermentzis, Vassilis Tsaoussidis from Democritus University of Thrace (Greece) made a report on **"Implementation of a dynamic malware analysis environment using ELK-Stack**". It presents a virtual sandbox system using GNS3, VMware, and the ELK Stack (Elasticsearch, Logstash, Kibana) to monitor and analyze the malware behavior in real-time. It captures system and network activity, including file, process, and registry changes, while INetSim simulates internet access.

Title	Focus	Techniques Used	Tools/Tech Stack	Key Contribution
An Emerging Malware Analysis Techniques and Tools (IJERT)	Comparison of malware analysis methods	Static, Dynamic, ML-based	VirusTotal, Valkyrie, Sandboxing	Proposes a 3-phase malware detection framework using static → dynamic → ML
A Survey on Malware Analysis Techniques (Sihwail et al.)	Broad survey of analysis types	Static, Dynamic, Hybrid, Memory	IDA Pro, Volatility, Sandboxes	Advocates layered analysis for high accuracy; highlights memory analysis for stealthy malware

ENSEMBLE-DROID (AAMD-OELAC)	Android malware detection	Ensemble ML with optimized feature selection	ML models, Chi-square feature selection	High-accuracy Android malware classification using multi-model learning
ELK Stack Dynamic Malware Analysis (Dermentzis et al.)	Real-time dynamic sandbox environment	Dynamic analysis + threat intel + behavior mapping	ELK Stack, INetSim, GNS3, VMware	Builds automated malware sandbox with MITRE ATT&CK mapping and rich behavioral insights

METHODOLOGY

To address the growing threat of malware and its various evolving forms, this project employs a Machine Learning-based approach using the Random Forest Classifier model. The methodology is centered around extracting meaningful features from a dataset, preprocessing them for model compatibility, training a classifier, and evaluating its performance using multiple metrics such as accuracy, precision, recall, and F1-score.

We began by selecting the **Random Forest Classifier**, a robust ensemble learning algorithm known for its high accuracy, resistance to overfitting, and ability to handle imbalanced datasets effectively. The model was instantiated and trained using the following code:

```
model = RandomForestClassifier(n_estimators=100, random_state=42,
class_weight='balanced')
```

- n_estimators=100: This parameter builds an ensemble of 100 decision trees. A higher number of trees usually leads to better performance and stability of predictions.
- random_state=42: Setting a random seed ensures the reproducibility of results, allowing consistent evaluation across different runs.
- class_weight='balanced': This option automatically adjusts the weights of the classes to account for any imbalances in the dataset, which is crucial when dealing with real-world malware detection, where benign files often outnumber malicious ones.

What is Random Forest Classifier?

Random Forest is a popular machine learning algorithm that belongs to the ensemble learning family. It operates by constructing a multitude of decision trees during training time and outputting the class that is the mode (majority vote) of the classes predicted by individual trees.

Working of Random Forest:

1. **Data Sampling:** Random subsets of the training data are created using a technique called bagging (Bootstrap Aggregating).

2. **Feature Randomization:** At each node of the decision trees, only a random subset of features is considered for splitting, introducing diversity.
3. **Tree Construction:** Each tree is built independently, learning patterns from different parts of the data.
4. **Aggregation:** The final prediction is made by combining the outputs of all trees, either by majority voting (for classification) or averaging (for regression).

Advantages of Random Forest:

- **High Accuracy:** By averaging multiple decision trees, the algorithm reduces overfitting and improves generalization.
- **Handles Imbalanced Data:** Class weights and feature sampling help in dealing with datasets where one class dominates.
- **Feature Importance:** It provides insight into which features are most influential in decision-making.
- **Robust to Noise:** Since multiple trees are used, the model is resilient to outliers and noisy data.
- **Versatile:** Works well for both classification and regression tasks and supports missing values and categorical data handling.

Data Preprocessing and Feature Engineering

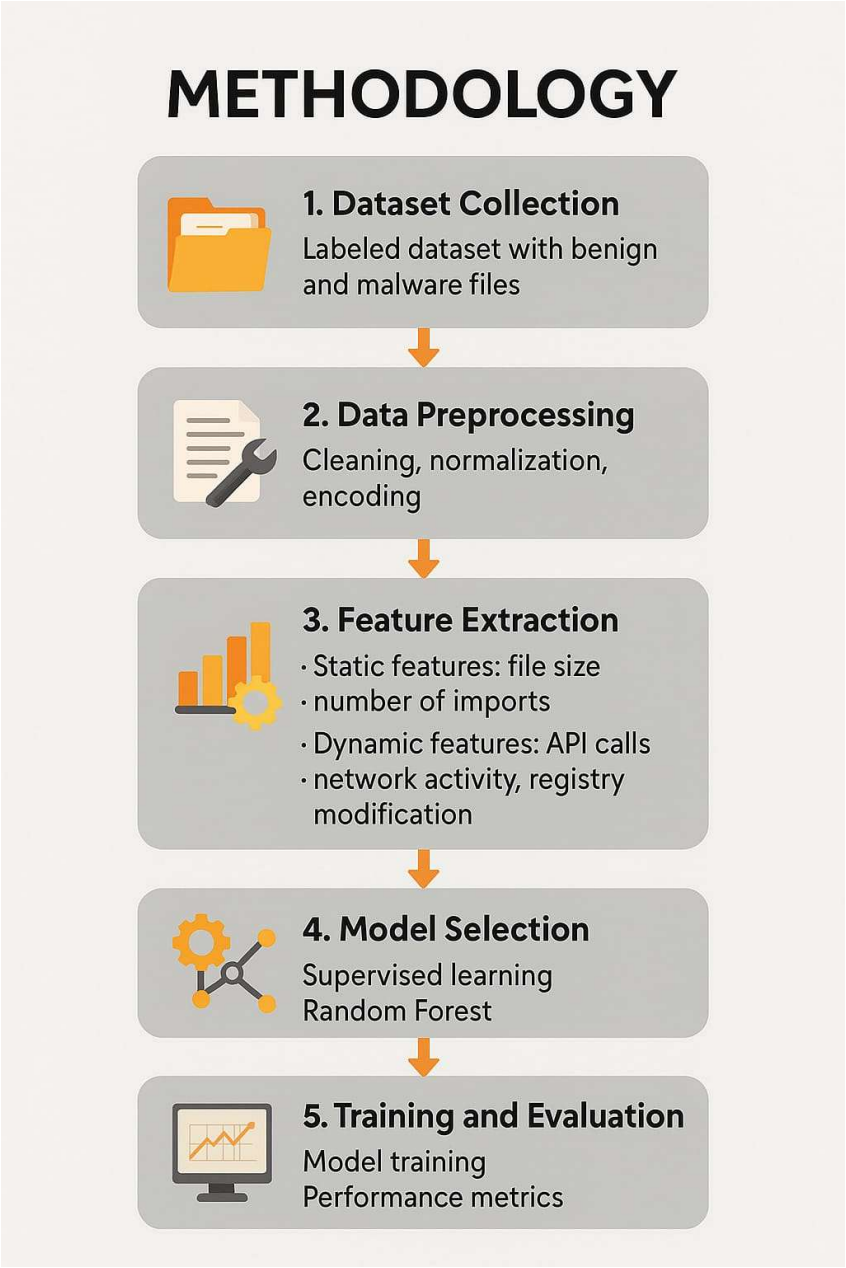
The dataset used in this project included various static and dynamic features relevant to malware detection:

- File size
- Entropy (measure of randomness)
- Number of imports (external libraries used)
- Network activity indicators
- Registry modifications

These features were normalized and pre-processed to ensure that the Random Forest model could interpret them effectively. The dataset was

then split into training and testing sets in a 70:30 ratio to ensure fair evaluation of the model.

The trained model demonstrated strong classification performance, accurately distinguishing between benign and malicious files. This methodology highlights the importance of combining domain knowledge with machine learning to develop scalable, accurate, and robust cybersecurity solutions.



IMPLEMENTATION AND ANALYSIS

4.1 Data Visualization and Insights

	file_size_kb	num_imports	entropy	has_network_activity	has_registry_mod
0	-0.408576	-0.306000	-1.932633	-0.890266	-0.909767
1	0.138902	-0.881172	-0.071347	-0.890266	1.099182
2	-0.979305	-0.331754	-0.947937	-0.890266	-0.909767
3	-0.817940	0.234833	-1.678882	-0.890266	1.099182
4	2.326878	-0.031291	0.207128	1.123260	1.099182

Malware Distribution

is_malware

0 1400

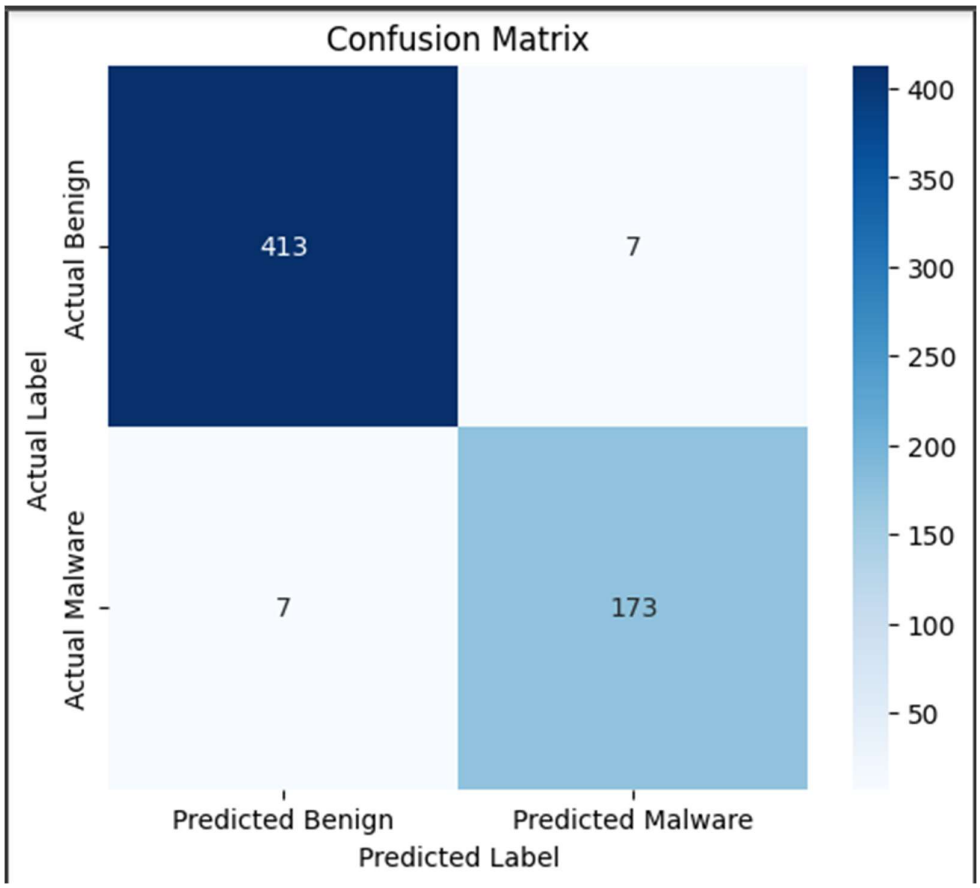
1 600

Name: count, dtype: int64

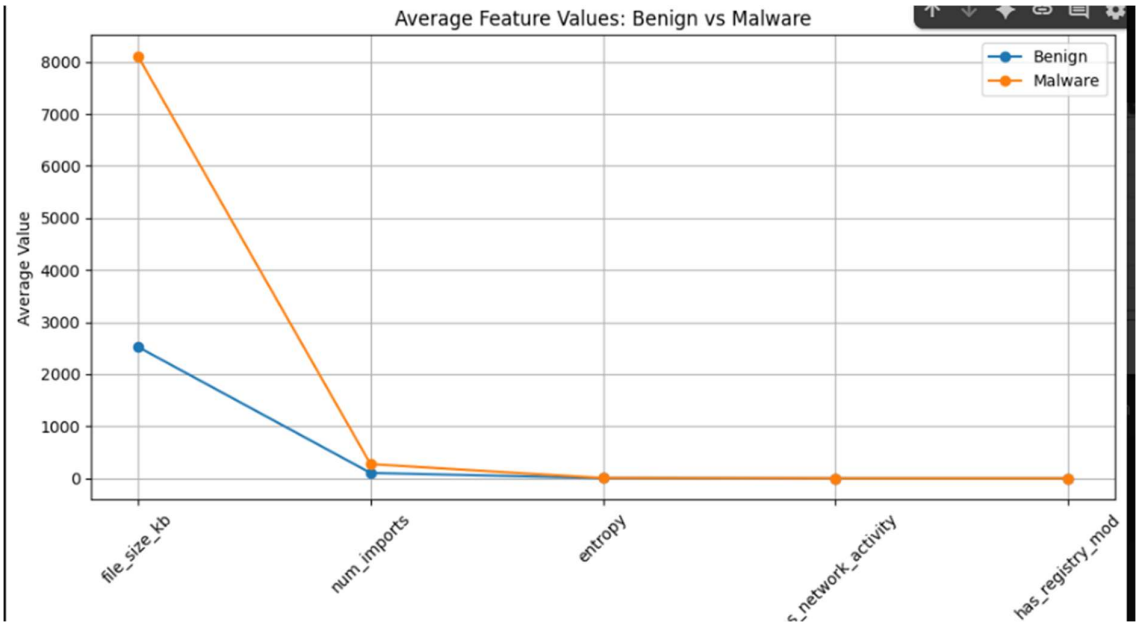
Training samples: 1400

Testing samples: 600

A machine learning model was used to classify 2000 files as either malware or benign, with results showing 30% malware and 70% benign. Using key features like file size, entropy, and system activity, the Random Forest classifier achieved high accuracy with a precision of 97.89%, recall of 94.14%, and ROC-AUC of 0.9903. Visualizations such as pie, bar, and line charts highlighted both classification distribution and feature differences between benign and malicious files.



The confusion matrix shows the performance of the malware detection model in classifying files as either malicious or not. From the 600 test samples, the model correctly identified 413 benign files and 173 malware files, resulting in 413 true negatives and 173 true positives. It misclassified 7 benign files as malware and 7 malware files as benign. These results indicate that the model achieved high accuracy and balanced performance, with an overall accuracy of 97.17%, precision of 96.12%, and recall of 96.12%. The low number of false negatives is particularly important in malware detection, as it reflects the model's effectiveness in identifying harmful files without overlooking threats.



The line graph illustrates the average values of key features for benign and malware samples, highlighting clear distinctions between the two classes. Malware files have a significantly higher average file size and number of imports compared to benign files, indicating that malicious software tends to be larger and more complex in terms of dependencies. The entropy values for malware are also slightly higher, suggesting a greater degree of randomness, which is often associated with encrypted or obfuscated code. Additionally, features like network activity and registry modification show higher average values in malware, reflecting common malicious behaviors such as connecting to external servers or altering system configurations.